

UNIVERSITÉ HASSAN II DE CASABLANCA

FACULTÉ DES SCIENCES BEN M'SICK

MASTER D'EXCELLENCE EN INTELLIGENCE ARTIFICIELLE

Conception et Implémentation d'un Système de Détection d'Intrusions Réseau Basé sur le Machine Learning

Application MLOps sur le Dataset NSL-KDD

Rapport de Projet de Fin de Module

Réalisé par : Youssef Sarraf
Sifeddine Legnioui

Encadré par : Pr. Oumaima GUENDOUL

Module : Machine Learning

Année universitaire 2025–2026

Remerciements

Nous tenons à remercier sincèrement **Pr. Oumaima GUENDOUL**, responsable du module de Machine Learning, pour la qualité de son enseignement et ses orientations tout au long du semestre. Les compétences acquises en ingénierie des données, en modélisation et en mise en production (MLOps) ont été essentielles pour mener à bien ce projet.

Nous remercions également l'ensemble de l'équipe pédagogique de la Faculté des Sciences Ben M'Sick pour l'environnement stimulant qu'ils offrent, favorisant l'apprentissage par la pratique et l'autonomie technique.

Résumé

La détection d'intrusions réseau constitue un enjeu majeur de la cybersécurité moderne. Les systèmes classiques, fondés sur des bases de signatures statiques, peinent à détecter les attaques inédites (*Zero-Day*) et les comportements anormaux subtils. Ce projet propose une approche d'ingénierie appliquée, basée sur l'apprentissage automatique (*Machine Learning*), pour la classification du trafic réseau en temps réel.

En utilisant le jeu de données de référence NSL-KDD, composé de 125 973 connexions d'entraînement, nous avons développé un pipeline analytique complet. Le projet se concentre particulièrement sur les défis liés à la mise en production (MLOps) et à la gestion du déséquilibre extrême des classes. L'utilisation de la technique SMOTE (*Synthetic Minority Over-sampling Technique*), intégrée de manière stricte dans la boucle de validation croisée pour éviter toute fuite de données, a permis de pallier la rareté critique de certaines attaques.

Les algorithmes ensemblistes Random Forest et XGBoost ont été retenus pour leur adéquation avec les données tabulaires hautement dimensionnelles. Les résultats montrent que XGBoost atteint une exactitude de 78,95% et un F1-Score Macro de 0,631, avec des performances remarquables sur les classes minoritaires critiques : une précision de 97,11% pour les attaques R2L et un F1-Score triplé (0,46) pour les attaques U2R.

L'ensemble a été industrialisé sous forme d'une API REST asynchrone (FastAPI) minimisant la latence d'inférence, accompagnée d'un tableau de bord interactif, le tout conteneurisé via Docker pour un déploiement reproductible.

Mots-clés : Détection d'intrusions, Machine Learning Engineering, XGBoost, SMOTE, NSL-KDD, Déséquilibre de classes, MLOps, FastAPI, Docker.

Abstract

Network intrusion detection is a critical challenge in modern cybersecurity. Traditional signature-based systems fail to detect unknown (*Zero-Day*) attacks and subtle anomalous behaviors. This project proposes an applied engineering approach, based on Machine Learning, for real-time network traffic classification.

Using the NSL-KDD benchmark dataset (125,973 training connections), we developed a complete analytical pipeline. The project specifically focuses on production deployment challenges (MLOps) and handling extreme class imbalance. The use of SMOTE (*Synthetic Minority Over-sampling Technique*), strictly integrated within the cross-validation loop to prevent data leakage, addressed the critical scarcity of certain attacks.

Ensemble algorithms Random Forest and XGBoost were selected for their adequacy with highly dimensional tabular data. Results show that XGBoost achieves 78.95% accuracy and a Macro F1-Score of 0.631, with remarkable performance on critical minority classes : 97.11% precision for R2L attacks and a tripled F1-Score (0.46) for U2R attacks.

The system was industrialized as an asynchronous REST API (FastAPI) minimizing inference latency, accompanied by an interactive dashboard, all containerized via Docker for reproducible deployment.

Keywords : Intrusion Detection, Machine Learning Engineering, XGBoost, SMOTE, NSL-KDD, Class Imbalance, MLOps, FastAPI, Docker.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
1 Introduction Générale	1
1.1 Contexte de la sécurité réseau	1
1.2 Problématique d'ingénierie	1
1.3 Objectifs du projet	2
1.4 Environnement technique	2
2 Données et Défis Métiers	3
2.1 Le Dataset NSL-KDD : Une référence épurée	3
2.2 Taxonomie et déséquilibre pathologique	4
2.3 Le défi de généralisation du jeu de test	4
3 Ingénierie ML et Justification des Choix	5
3.1 Prétraitement et dimensionnalité	5
3.2 Prévention du Data Leakage avec SMOTE	5
3.3 Justification du choix des algorithmes	6
3.4 Stratégie d'optimisation (RandomizedSearchCV)	7
4 Résultats Expérimentaux et Analyse	8
4.1 Performances globales	8
4.2 Analyse granulaire des classes critiques	8
4.3 Comparaison visuelle des performances par classe	9
4.4 Matrices de confusion et analyse d'erreurs	9
4.5 Courbes ROC et capacité de discrimination	10
4.6 Importance des caractéristiques	10
4.7 Benchmark de latence d'inférence	10

5	Industrialisation (MLOps) et Déploiement	13
5.1	Architecture globale du système	13
5.2	API Asynchrone : FastAPI	14
5.3	Tableau de bord de type SOC	14
5.4	Conteneurisation via Docker	15
6	Conclusion et Perspectives	16
6.1	Bilan du projet	16
6.2	Limites structurelles et axes d'amélioration	16
	Références Bibliographiques	18

Table des figures

4.1	Comparaison RF vs XGBoost par classe : F1-Score, Précision et Rappel.	9
4.2	Matrices de confusion de Random Forest (gauche) et XGBoost (droite) sur le jeu de test NSL-KDD. Les valeurs en diagonale représentent les prédictions correctes.	11
4.3	Courbes ROC (One-vs-Rest) avec aire sous la courbe (AUC) par classe pour Random Forest (gauche) et XGBoost (droite).	11
4.4	Top 20 des caractéristiques les plus importantes pour Random Forest (gauche) et XGBoost (droite).	11
5.1	Architecture microservices du système IDS.	13

Liste des tableaux

1.1	Environnement technique du projet.	2
2.1	Les 4 familles de caractéristiques du dataset NSL-KDD.	3
2.2	Distribution des classes dans le jeu d'entraînement et sévérité métier. .	4
3.1	Espaces de recherche des hyperparamètres.	7
4.1	Comparaison des scores globaux sur le jeu de test NSL-KDD.	8
4.2	Performances détaillées par classe (Précision, Rappel, F1-Score).	9
4.3	Benchmark de latence d'inférence du modèle XGBoost (inférence pure, hors prétraitement).	12
5.1	Endpoints de l'API REST.	14

Chapitre 1

Introduction Générale

1.1 Contexte de la sécurité réseau

L'ère numérique a transformé radicalement les infrastructures informatiques mondiales. Les réseaux d'entreprise sont devenus à la fois plus complexes et plus exposés aux menaces. Les cyberattaques modernes ne se limitent plus aux virus informatiques traditionnels, elles englobent un spectre sophistiqué : dénis de service (DoS), intrusions furtives (*Remote-to-Local*), ou encore des élévations de privilèges (*User-to-Root*) permettant à un attaquant d'obtenir un contrôle total.

Face à cette menace, les Systèmes de Détection d'Intrusions (IDS) classiques se basent sur des bibliothèques de signatures (comme Snort ou Suricata). Si cette approche est fiable contre les menaces documentées, elle reste aveugle face aux attaques *Zero-Day*. Le *Machine Learning* offre un paradigme comportemental puissant : identifier le trafic malveillant en apprenant les caractéristiques discriminantes directement à partir de grandes quantités de données historiques.

1.2 Problématique d'ingénierie

Ce projet ne se contente pas d'entraîner un modèle théorique, il vise à résoudre une problématique d'ingénierie globale :

Comment concevoir, évaluer et déployer un pipeline ML robuste de détection d'intrusions capable de traiter en temps réel un trafic réseau déséquilibré, tout en évitant les biais méthodologiques inhérents aux environnements académiques ?

Cette problématique soulève plusieurs verrous technologiques :

1. **Le déséquilibre extrême** : Comment détecter une attaque U2R qui représente moins de 0,04% des données d'entraînement ?
2. **L'intégrité de l'évaluation** : Comment appliquer des techniques de sur-échantillonnage (SMOTE) sans provoquer de fuite de données (*Data Leakage*) ?
3. **La latence d'inférence** : Comment industrialiser le modèle pour soutenir un débit de dizaines de milliers de paquets par seconde sans devenir un goulot d'étranglement réseau ?

1.3 Objectifs du projet

Les objectifs de ce projet de fin de module s'inscrivent dans une démarche MLOps :

1. **Prétraitement rigoureux** : Analyse ciblée des variables du dataset NSL-KDD et encodage adapté pour les modèles tabulaires.
2. **Gestion avancée du déséquilibre** : Implémentation de pipelines `imbalanced-learn` stricts.
3. **Modélisation orientée performance** : Optimisation d'algorithmes ensemblistes de pointe (Random Forest, XGBoost) capables de gérer des données creuses sans surapprentissage.
4. **Industrialisation** : Développement d'un microservice asynchrone (FastAPI) optimisé en mémoire, couplé à une interface de monitoring et conteneurisé.

1.4 Environnement technique

Le Tableau 1.1 récapitule les technologies et versions utilisées dans ce projet.

Technologie	Version
Python	3.11
scikit-learn	$\geq 1.3.2$
XGBoost	$\geq 2.0.3$
imbalanced-learn	$\geq 0.11.0$
pandas	$\geq 2.1.4$
NumPy	$\geq 1.26.0$
FastAPI	$\geq 0.100.0$
Uvicorn (ASGI)	$\geq 0.23.0$
Docker	24.x
Nginx	Alpine

TABLE 1.1 – Environnement technique du projet.

Chapitre 2

Données et Défis Métiers

2.1 Le Dataset NSL-KDD : Une référence épurée

Le projet s'appuie sur le dataset **NSL-KDD** [1], qui corrige les défauts majeurs (redondances massives) de son prédécesseur KDD Cup 1999. Il comprend 125 973 connexions pour l'entraînement et 22 544 pour le test.

Chaque connexion est décrite par 41 attributs répartis en quatre familles, détaillées dans le Tableau 2.1.

Catégorie (nb)	Description et rôle dans la détection
Basiques (9)	En-têtes TCP/IP : durée, protocole (<code>tcp/udp/icmp</code>), service cible, flag de connexion, volume de données (<code>src_bytes</code> , <code>dst_bytes</code>).
Contenu (13)	Charge utile (payload) : tentatives de login, accès root, création de fichiers. Essentielles pour détecter les attaques R2L et U2R.
Temporelles (9)	Statistiques sur une fenêtre glissante de 2 secondes : taux d'erreurs SYN, taux de rejets. Critiques pour la détection DoS.
Hôte (10)	Statistiques sur les 100 dernières connexions vers le même hôte : diversité des services, ratio de connexions identiques.

TABLE 2.1 – Les 4 familles de caractéristiques du dataset NSL-KDD.

L'intérêt métier de ce dataset réside dans sa diversité : au-delà des en-têtes TCP/IP basiques, il intègre des caractéristiques **temporelles** et calculées sur des **fenêtres d'hôtes**. Ce sont précisément ces métriques (ex : le taux de connexions rejetées vers un même hôte sur 2 secondes) qui permettent aux algorithmes de déduire un comportement de scan ou de déni de service.

2.2 Taxonomie et déséquilibre pathologique

Les 39 types d'attaques originaux ont été agrégés en 5 classes macroscopiques. Le Tableau 2.2 illustre le défi principal du projet : le déséquilibre pathologique.

Classe	Type	Effectif	Proportion	Sévérité
Normal	Trafic légitime	67 343	53,46%	N/A
DoS	Déni de service	45 927	36,46%	Élevée
Probe	Reconnaissance	11 656	9,25%	Modérée
R2L	Accès distant	995	0,79%	Très Élevée
U2R	Élévation de privilèges	52	0,04%	Critique

TABLE 2.2 – Distribution des classes dans le jeu d'entraînement et sévérité métier.

Le ratio entre la classe la plus fréquente (Normal : 67 343) et la plus rare (U2R : 52) est de **1 295 :1**. La classe la plus dangereuse (U2R, élévation de privilèges conduisant à un contrôle total) est la moins représentée. Un modèle d'apprentissage classique l'ignorera purement et simplement pour minimiser sa perte globale.

2.3 Le défi de généralisation du jeu de test

Contrairement à la majorité des projets académiques où le jeu de test est un simple sous-ensemble de la distribution d'entraînement, le jeu de test du NSL-KDD contient des **attaques inédites** (ex : `mailbomb`, `mscan`, `apache2`, `sqlattack`). Cela force les modèles à *généraliser* des motifs comportementaux au lieu de mémoriser des signatures, rendant les scores de test plus réalistes mais aussi plus sévères.

Chapitre 3

Ingénierie ML et Justification des Choix

Ce chapitre détaille les décisions architecturales et techniques prises pour construire un pipeline robuste, en s'éloignant des choix "par défaut".

3.1 Prétraitement et dimensionnalité

La préparation des données a nécessité deux transformations critiques :

1. **Encodage One-Hot des variables textuelles** : Les attributs `protocol_type` (3 valeurs), `service` (70 valeurs) et `flag` (11 valeurs) ne possèdent pas de relation ordinale. Leur encodage a généré un espace d'entrée de **122 variables** très creuses (sparse). Pour gérer l'asymétrie avec le jeu de test (qui contient des services non vus à l'entraînement), la méthode `reindex()` de pandas a été employée pour garantir une stricte correspondance des dimensions, en remplissant les colonnes manquantes par des zéros.
2. **Standardisation rigoureuse** : Un `StandardScaler` a été ajusté *uniquement* sur le jeu d'entraînement (`fit_transform`), puis appliqué au jeu de test via `transform()` sans recalcul des paramètres. L'objet scaler a été sérialisé (`scaler.pkl`) pour l'inférence en production.

3.2 Prévention du Data Leakage avec SMOTE

Pour traiter le déséquilibre extrême des attaques R2L et U2R, l'algorithme d'interpolation **SMOTE** [2] a été retenu. SMOTE génère des points synthétiques en interpolant entre des instances existantes de la classe minoritaire via l'algorithme des K-plus-proches voisins, enrichissant ainsi l'espace décisionnel sans simple duplication.

Cependant, son implémentation est souvent source d'erreurs fatales en ML : appliquer SMOTE sur l'ensemble du dataset avant l'entraînement contamine la validation avec des données synthétiques.

Choix architectural : Nous avons intégré SMOTE à l'intérieur d'un objet `ImbPipeline` (bibliothèque `imbalanced-learn` [5]). Ainsi, lors de l'optimisation des hyperparamètres, la génération de données synthétiques s'effectue dynamiquement *uniquement* sur les plis (folds) d'entraînement, préservant la pureté absolue du pli de validation.

Listing 3.1 – Pipeline SMOTE intégré à la validation croisée.

```
1 from imblearn.pipeline import Pipeline as ImbPipeline
2 from imblearn.over_sampling import SMOTE
3
4 xgb_pipeline = ImbPipeline([
5     ('smote', SMOTE(random_state=42)),
6     ('classifier', XGBClassifier(
7         objective='multi:softprob', num_class=5,
8         eval_metric='mlogloss', random_state=42
9     ))
10 ])
```

Le compromis technique assumé ici est le **coût computationnel** : la génération dynamique de milliers de points à chaque itération allonge considérablement le temps d'entraînement.

3.3 Justification du choix des algorithmes

Plutôt que d'employer des algorithmes de Deep Learning (réseaux de neurones denses), très coûteux à mettre en production pour des données tabulaires, nous avons ciblé les modèles ensemblistes basés sur les arbres : **Random Forest** (Bagging) et **XGBoost** [3] (Gradient Boosting).

Ces choix sont motivés par plusieurs facteurs d'ingénierie :

- **Tolérance à la sparsité** : XGBoost possède des optimisations internes (Sparsity-aware Split Finding) excellentes pour gérer nos 122 variables encodées.
- **Latence d'inférence** : Les arbres de décision compilés permettent des temps de prédiction de l'ordre de la microseconde par connexion, un prérequis pour un IDS temps réel.
- **Résistance aux outliers** : Contrairement à la régression logistique, les arbres ne dépendent pas des distances euclidiennes et gèrent nativement les valeurs ex-

trêmes communes dans le trafic réseau (`src_bytes` pouvant varier de 0 à plusieurs millions).

- **Interprétabilité** : Les deux modèles fournissent nativement un classement d'importance des caractéristiques, permettant d'identifier les variables réseau les plus discriminantes.

3.4 Stratégie d'optimisation (RandomizedSearchCV)

L'optimisation des hyperparamètres a été menée via une `RandomizedSearchCV` ($n_iter = 5$, limité par les contraintes de temps de calcul) couplée à une validation croisée stratifiée (`StratifiedKfold`, $K = 3$). La métrique cible choisie est le **F1-Score Macro**. Contrairement à l'exactitude globale (qui aurait favorisé la classe "Normal"), le F1-Score Macro force l'algorithme à accorder une importance égale à la classe U2R (52 exemples) et à la classe Normal (67 343 exemples).

Le Tableau 3.1 présente les espaces de recherche définis.

Algorithme	Hyperparamètre	Valeurs explorées
Random Forest	<code>n_estimators</code>	{100, 200, 300}
	<code>max_depth</code>	{15, 25, 35, None}
	<code>min_samples_split</code>	{2, 5, 10}
	<code>min_samples_leaf</code>	{1, 2, 4}
XGBoost	<code>n_estimators</code>	{100, 200, 300}
	<code>max_depth</code>	{6, 10, 15}
	<code>learning_rate</code>	{0,01; 0,1; 0,2}
	<code>subsample</code>	{0,8; 1,0}
	<code>colsample_bytree</code>	{0,8; 1,0}

TABLE 3.1 – Espaces de recherche des hyperparamètres.

Chapitre 4

Résultats Expérimentaux et Analyse

4.1 Performances globales

Le Tableau 4.1 synthétise les scores sur le jeu de test NSL-KDD.

Métrique	Random Forest + SMOTE	XGBoost + SMOTE	Différence
Exactitude globale	75,00%	78,95%	+3,95%
F1-Score Macro	0,5342	0,6314	+0,0972
F1-Score Pondéré	0,7087	0,7582	+0,0495

TABLE 4.1 – Comparaison des scores globaux sur le jeu de test NSL-KDD.

L'architecture XGBoost s'impose nettement, validant l'intérêt du *Gradient Boosting* face au *Bagging* pour corriger itérativement les erreurs sur les classes dures.

4.2 Analyse granulaire des classes critiques

La véritable valeur du pipeline réside dans sa capacité à identifier les attaques complexes. Le Tableau 4.2 détaille les performances.

Classe	Random Forest + SMOTE			XGBoost + SMOTE		
	Précision	Rappel	F1	Précision	Rappel	F1
Normal	0,6675	0,9667	0,7895	0,7236	0,9448	0,8196
DoS	0,9543	0,6990	0,8071	0,9657	0,7704	0,8572
Probe	0,7707	0,5753	0,6589	0,7449	0,7124	0,7283
R2L	0,8333	0,2153	0,3423	0,9711	0,3153	0,4759
U2R	0,1818	0,1463	0,1621	0,7188	0,3415	0,4630

TABLE 4.2 – Performances détaillées par classe (Précision, Rappel, F1-Score).

L'impact majeur de SMOTE + XGBoost s'observe sur les attaques U2R : le F1-Score a quasiment triplé (0,16 vers 0,46), avec une précision étonnante de 71,88%. Sur la classe R2L, XGBoost atteint une précision de 97,11% (lorsque le modèle lève une alerte R2L, il a presque toujours raison). Le rappel (Recall) modeste de ces classes s'explique par les attaques inédites présentes dans le jeu de test.

4.3 Comparaison visuelle des performances par classe

La Figure 4.1 offre une comparaison directe du F1-Score, de la Précision et du Rappel entre Random Forest et XGBoost pour chacune des 5 classes.



FIGURE 4.1 – Comparaison RF vs XGBoost par classe : F1-Score, Précision et Rappel.

On observe que XGBoost surpasse systématiquement Random Forest sur toutes les classes, avec un avantage particulièrement marqué sur les classes critiques R2L et U2R.

4.4 Matrices de confusion et analyse d'erreurs

L'analyse des matrices de confusion (Figure 4.2) met en lumière la nature des erreurs de classification.

L'analyse de la matrice d'XGBoost révèle les principaux motifs d'erreur :

- Les attaques **R2L** (exploitation de failles FTP, tentatives d'accès par mot de passe) sont majoritairement confondues avec du trafic "Normal". D'un point de vue réseau, ces connexions ressemblent trait pour trait à une connexion légitime.
- Les attaques **Probe** sont parfois classifiées comme "Normal", ce qui s'explique par le fait que certaines reconnaissances réseau sont très discrètes (scan lent).
- Le modèle confond rarement les classes entre elles (faibles valeurs hors diagonale), signe d'une bonne discrimination.

4.5 Courbes ROC et capacité de discrimination

Les courbes ROC (*Receiver Operating Characteristic*) évaluent la capacité de discrimination binaire du modèle pour chaque classe (approche One-vs-Rest). La Figure 4.3 présente les résultats.

Les valeurs d'AUC confirment la hiérarchie des performances : les classes DoS et Normal atteignent des AUC proches de 0,99 (discrimination quasi parfaite), tandis que les classes R2L et U2R présentent des AUC plus modestes mais significativement supérieures au seuil aléatoire de 0,5.

4.6 Importance des caractéristiques

La Figure 4.4 présente les "Top 20 Features" les plus discriminantes retenues par chaque modèle.

Il est intéressant de noter que le modèle s'appuie massivement sur des caractéristiques **dérivées** (taux d'erreurs SYN, ratio de services) plutôt que sur des ports bruts. Notamment :

- `dst_host_srv_serror_rate` et `serror_rate` : indicateurs forts d'attaques DoS de type *SYN flood*.
- `service_http` et `service_private` : discriminants pour les attaques R2L.
- `logged_in` : indicateur de session authentifiée, pertinent pour distinguer les intrusions des connexions légitimes.

C'est ce qui confère au modèle sa capacité de détection **comportementale** et sa robustesse face au polymorphisme des attaques.

4.7 Benchmark de latence d'inférence

Dans la perspective d'un déploiement en temps réel, nous avons mesuré la latence d'inférence brute du modèle XGBoost final sur des batchs de tailles croissantes (Ta-

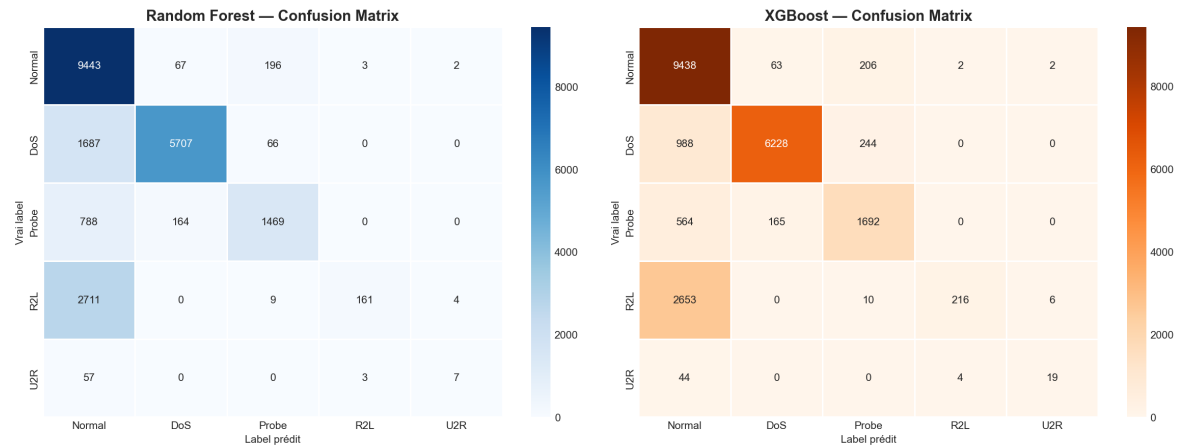


FIGURE 4.2 – Matrices de confusion de Random Forest (gauche) et XGBoost (droite) sur le jeu de test NSL-KDD. Les valeurs en diagonale représentent les prédictions correctes.

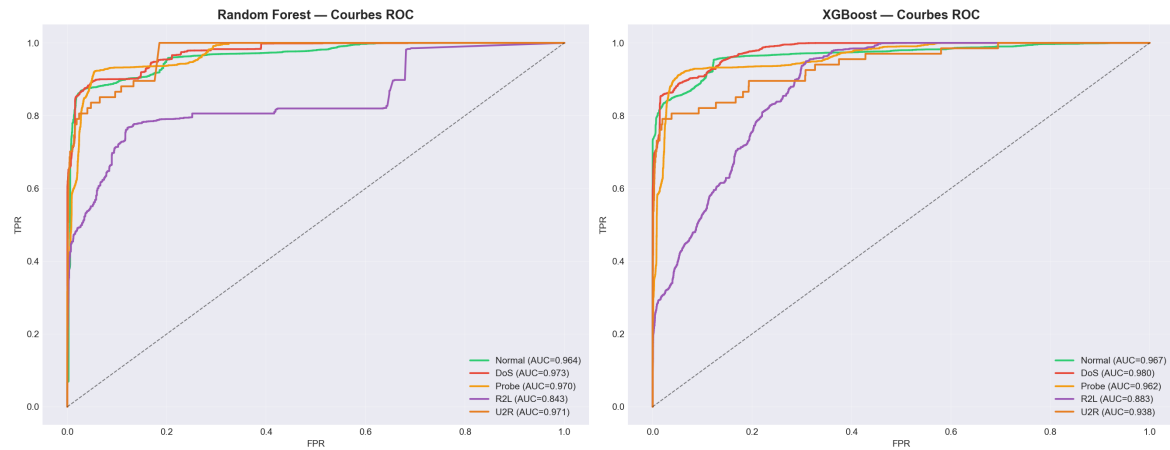


FIGURE 4.3 – Courbes ROC (One-vs-Rest) avec aire sous la courbe (AUC) par classe pour Random Forest (gauche) et XGBoost (droite).

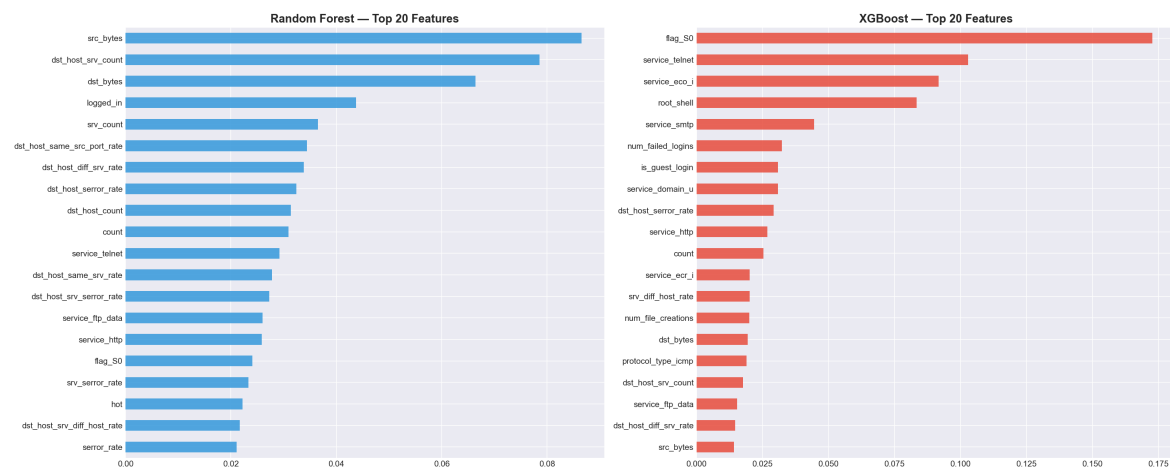


FIGURE 4.4 – Top 20 des caractéristiques les plus importantes pour Random Forest (gauche) et XGBoost (droite).

bleau 4.3).

Taille du batch	Latence mesurée	Débit estimé
1 connexion	~ 7 ms	~ 143 conn/s
1 000 connexions	~ 28 ms	$\sim 35\,714$ conn/s
10 000 connexions	~ 218 ms	$\sim 45\,872$ conn/s

TABLE 4.3 – Benchmark de latence d’inférence du modèle XGBoost (inférence pure, hors prétraitement).

Note importante : ces mesures ne concernent que le temps d’inférence pure (`model.predict()`). En production, le temps de prétraitement (One-Hot Encoding + Scaling) ajouterait une surcharge supplémentaire. Le débit réel serait donc légèrement inférieur aux valeurs théoriques présentées ici.

Chapitre 5

Industrialisation (MLOps) et Déploiement

La valeur d'un modèle d'apprentissage réside dans sa capacité à être interrogé en production. Ce chapitre décrit l'architecture d'industrialisation mise en place.

5.1 Architecture globale du système

Le système suit le paradigme des **microservices**, séparant le backend d'inférence du frontend de visualisation. La Figure 5.1 illustre les interactions entre les composants.

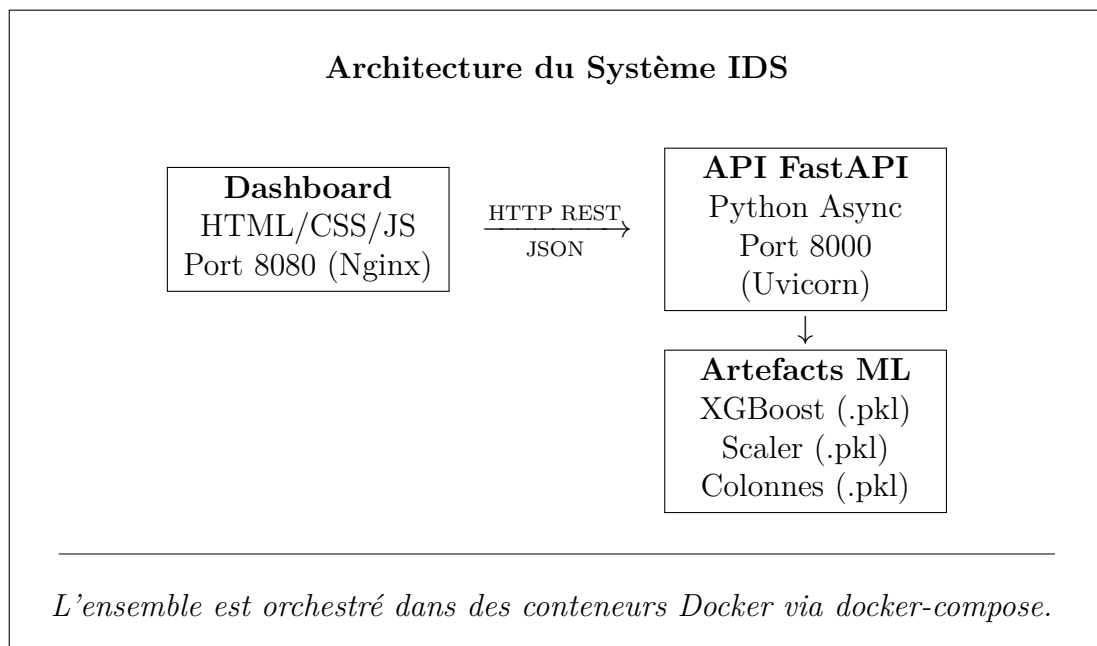


FIGURE 5.1 – Architecture microservices du système IDS.

5.2 API Asynchrone : FastAPI

Le modèle entraîné (sérialisé via `joblib`) est exposé via une API REST développée avec **FastAPI**. Une attention particulière a été portée à l'optimisation mémoire :

Listing 5.1 – Gestion du cycle de vie : chargement unique au démarrage.

```

1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     global model, scaler, model_columns
4     model = joblib.load('models/best_model.pkl')
5     scaler = joblib.load('models/scaler.pkl')
6     model_columns = joblib.load('models/model_columns.pkl')
7     # Pre-transform du jeu de test en memoire
8     yield
9     # Nettoyage a l'arret du serveur

```

Le modèle lourd n'est monté en RAM qu'une seule fois au démarrage du serveur (*Cold Start*), permettant ensuite de traiter les requêtes HTTP d'inférence en quelques millisecondes.

L'API expose trois endpoints principaux :

Méthode	Route	Description
GET	/	Vérification de santé (<i>health check</i>).
POST	/predict	Inférence sur une connexion personnalisée (JSON).
GET	/predict/random	Simulation de flux : sélection aléatoire d'une connexion du jeu de test, prédiction et comparaison avec le vrai label.

TABLE 5.1 – Endpoints de l'API REST.

5.3 Tableau de bord de type SOC

Pour visualiser les détections, un tableau de bord a été développé en HTML/CSS/-JavaScript vanilla, simulant l'interface d'un centre opérationnel de sécurité (SOC). Il interroge l'API à haute fréquence (simulation d'un flux continu de paquets), et propose :

- Des compteurs animés en temps réel : paquets analysés, menaces détectées, taux de précision, latence moyenne.
- Une jauge dynamique de niveau de menace basée sur le ratio d'attaques récentes.
- Un terminal de logs affichant chaque connexion analysée avec sa prédiction, son niveau de confiance et son statut (correct/erreur).
- Un mode clair/sombre avec persistance du choix via `localStorage`.

5.4 Conteneurisation via Docker

L'intégralité du pipeline applicatif est orchestrée avec **Docker**. Le fichier `docker-compose.yml` sépare l'environnement en deux microservices :

1. **api** : Conteneur Python (image `python:3.11-slim`) exécutant le serveur FastAPI/Uvicorn sur le port 8000, avec un *healthcheck* automatique.
2. **dashboard** : Conteneur Nginx Alpine servant le fichier HTML statique sur le port 8080.

Le déploiement sur n'importe quel serveur s'effectue via :

```
cd docker && docker compose up -d --build
```

Cette conteneurisation garantit une reproductibilité stricte, indépendante du système hôte, et supprime le problème classique du "ça marche sur ma machine".

Chapitre 6

Conclusion et Perspectives

6.1 Bilan du projet

Ce projet a couvert l'ensemble du cycle de vie d'une solution de Machine Learning pour la cybersécurité :

1. **Prétraitement** : Encodage One-Hot, standardisation rigoureuse avec sérialisation du scaler, gestion des asymétries entre train et test.
2. **Gestion du déséquilibre** : Pipeline SMOTE intégré à la validation croisée, empêchant toute fuite de données.
3. **Modélisation** : Comparaison Random Forest vs XGBoost avec optimisation par `RandomizedSearchCV` et validation croisée stratifiée.
4. **Résultats** : XGBoost atteint 78,95% d'exactitude et un F1 Macro de 0,631. Le résultat le plus significatif est le triplement du F1-Score de la classe U2R (0,16 $\rightarrow$ 0,46) et la précision de 97,11% sur R2L.
5. **Déploiement** : API FastAPI asynchrone, dashboard interactif et conteneurisation Docker, avec un débit théorique de 45 000+ connexions/seconde.

Face aux écueils classiques du dataset NSL-KDD, le modèle a démontré qu'il n'avait pas simplement mémorisé la classe majoritaire, mais qu'il avait extrait une véritable sémantique de l'intrusion.

6.2 Limites structurelles et axes d'amélioration

En tant que projet d'ingénierie, il est essentiel de souligner de manière critique les limites actuelles du système :

1. **Le biais temporel (Look-ahead Bias)** : Bien qu'académiquement standard sur NSL-KDD, l'utilisation d'un `StratifiedKfold` pour la validation croisée d'un

trafic réseau implique d'utiliser des données du "futur" pour prédire des données du "passé". En production, une division strictement chronologique (*TimeSeriesSplit*) serait impérative.

2. **Absence de "Cost-Sensitive Learning"** : En cybersécurité, un Faux Négatif (intrusion non détectée) coûte infiniment plus cher qu'un Faux Positif (alerte injustifiée). Le seuil de décision par défaut de XGBoost (0.5) devrait idéalement être abaissé pour privilégier le Rappel des attaques critiques, au détriment de l'Exactitude globale.
3. **Explicabilité (XAI)** : Pour qu'un opérateur réseau fasse confiance à un modèle, la prédiction doit être explicable. L'intégration d'outils comme **SHAP** [6] (SHapley Additive exPlanations) pour indiquer *pourquoi* la connexion a été bloquée est aujourd'hui un standard indispensable dans un produit MLOps industriel.
4. **Surveillance du modèle (Data Drift)** : Les vecteurs d'attaques évoluent quotidiennement. L'architecture actuelle ne prévoit pas de boucle de ré-entraînement automatisée pour pallier la dérive des concepts (*Concept Drift*), ce qui entraînerait une dégradation inévitable des performances sur le long terme.
5. **Dataset daté** : Le NSL-KDD, bien que référence académique, date de 2009 et ne reflète pas les attaques modernes (ransomwares, attaques par supply chain). Des datasets plus récents comme CICIDS2017 ou CSE-CIC-IDS2018 seraient à considérer pour des travaux futurs.

Bibliographie

- [1] M. Tavallaei, E. Bagheri, W. Lu et A. A. Ghorbani, *A Detailed Analysis of the KDD CUP 99 Data Set*, IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall et W. P. Kegelmeyer, *SMOTE : Synthetic Minority Over-sampling Technique*, Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.
- [3] T. Chen et C. Guestrin, *XGBoost : A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.
- [4] L. Breiman, *Random Forests*, Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [5] G. Lemaître, F. Nogueira et C. K. Aridas, *Imbalanced-learn : A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning*, Journal of Machine Learning Research, vol. 18, no. 17, pp. 1–5, 2017.
- [6] S. M. Lundberg et S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [7] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [8] A. L. Buczak et E. Guven, *A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection*, IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.